

Name: Kanika Saini

SAP ID: 590034098

Program: BTech CSE B16

HDOC Day-9 – C Programming

Question 1: Convert Seconds into Days, Hours, Minutes and Seconds

1. Problem Analysis

Problem Statement:

Write a C program that accepts time in seconds and converts it into **days, hours, minutes, and seconds**.

Input:

Total time in seconds.

Conversion:

- 1 day = 86400 seconds
- 1 hour = 3600 seconds
- 1 minute = 60 seconds

2. Test Case Table

Test Case	Input (seconds)	Expected Output	Actual Output	Result
1	90061	1 day, 1 hour, 1 minute, 1 second	1 day, 1 hour, 1 minute, 1 second	Pass
2	3665	0 days, 1 hour, 1 minute, 5 seconds	0 days, 1 hour, 1 minute, 5 seconds	Pass
3	172800	2 days, 0 hours, 0 minutes, 0 seconds	2 days, 0 hours, 0 minutes, 0 seconds	Pass

3. Algorithm

Step 1: Start.

Step 2: Declare variables `total_seconds`, `days`, `hours`, `minutes`, and `seconds`.

Step 3: Input total time in seconds.

Step 4: Calculate days:

```
days = total_seconds / 86400
```

Step 5: Find the remaining seconds:

```
total_seconds = total_seconds % 86400
```

Step 6: Calculate hours:

```
hours = total_seconds / 3600
```

Step 7: Find the remaining seconds:

```
total_seconds = total_seconds % 3600
```

Step 8: Calculate minutes:

```
minutes = total_seconds / 60
```

Step 9: Calculate remaining seconds:

```
seconds = total_seconds % 60
```

Step 10: Display days, hours, minutes, and seconds.

Step 11: Stop.

4. Pseudocode

BEGIN

DECLARE `total_seconds`, `days`, `hours`, `minutes`, `seconds`

INPUT `total_seconds`

`days` ← `total_seconds` / 86400

`total_seconds` ← `total_seconds` % 86400

`hours` ← `total_seconds` / 3600

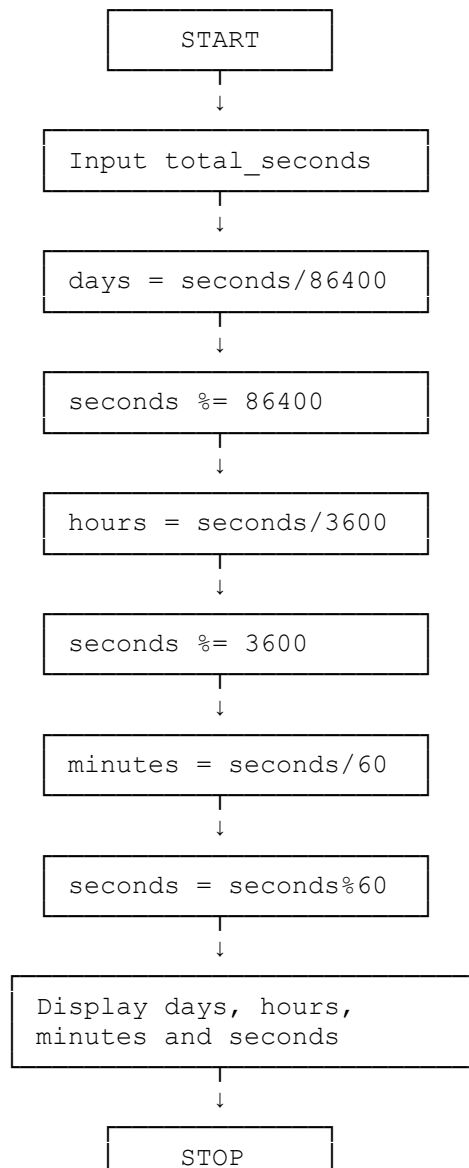
`total_seconds` ← `total_seconds` % 3600

```
minutes ← total_seconds / 60
seconds ← total_seconds % 60

DISPLAY days, hours, minutes, seconds

END
```

5. Flowchart



6. C Program

```
#include <stdio.h>

int main()
{
    int total_seconds, days, hours, minutes, seconds;

    printf("Enter time in seconds: ");
    scanf("%d", &total_seconds);

    days = total_seconds / 86400;
    total_seconds = total_seconds % 86400;

    hours = total_seconds / 3600;
    total_seconds = total_seconds % 3600;

    minutes = total_seconds / 60;
    seconds = total_seconds % 60;

    printf("Days = %d\n", days);
    printf("Hours = %d\n", hours);
    printf("Minutes = %d\n", minutes);
    printf("Seconds = %d\n", seconds);

    return 0;
}
```

7. Compile and Execute in Kali Linux

Save the program as `time.c`.

```
cc time.c
./a.out
```

```
(kali㉿kali)-[~/Desktop]
$ cc time.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter time in seconds: 90061
Days = 1
Hours = 1
Minutes = 1
Seconds = 1

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter time in seconds: 3665
Days = 0
Hours = 1
Minutes = 1
Seconds = 5

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter time in seconds: 172800
Days = 2
Hours = 0
Minutes = 0
Seconds = 0
```

8. Evaluation Against Prepared Test Cases

Test Case 1: Input = 90061

Program output = 1 day, 1 hour, 1 minute, 1 second

Expected output matches actual output. **PASS ✓**

Test Case 2: Input = 3665

Program output = 0 days, 1 hour, 1 minute, 5 seconds

Expected output matches actual output. **PASS ✓**

Test Case 3: Input = 172800

Program output = 2 days, 0 hours, 0 minutes, 0 seconds

Expected output matches actual output. **PASS ✓**

Question 2: Check Whether a Year is a Leap Year

1. Problem Analysis

Problem Statement:

Write a C program to input a year and determine whether it is a **leap year or not**.

Input:

A year.

Condition:

A year is a leap year when:

- It is divisible by **400**, OR
- It is divisible by **4 but not by 100**.

For example:

- 2024 → Leap year
 - 1900 → Not a leap year
 - 2000 → Leap year
-

2. Test Case Table

Test Case	Input Year	Expected Output	Actual Output	Result
1	2024	Leap Year	Leap Year	Pass
2	1900	Not a Leap Year	Not a Leap Year	Pass
3	2000	Leap Year	Leap Year	Pass

These cases are useful because they test both ordinary leap years and the special century-year rule.

3. Algorithm

Step 1: Start.

Step 2: Declare integer variable `year`.

Step 3: Input the year from the user.

Step 4: Check whether `year % 400 == 0`.

Step 5: If true, declare it a leap year.

Step 6: Otherwise, check whether `year % 100 == 0`.

Step 7: If true, declare it not a leap year.

Step 8: Otherwise, check whether `year % 4 == 0`.

Step 9: If true, declare it a leap year.

Step 10: Otherwise, declare it not a leap year.

Step 11: Stop.

4. Pseudocode

```
BEGIN

DECLARE year

INPUT year

IF year % 400 = 0 THEN
    DISPLAY "Leap Year"

ELSE IF year % 100 = 0 THEN
    DISPLAY "Not a Leap Year"

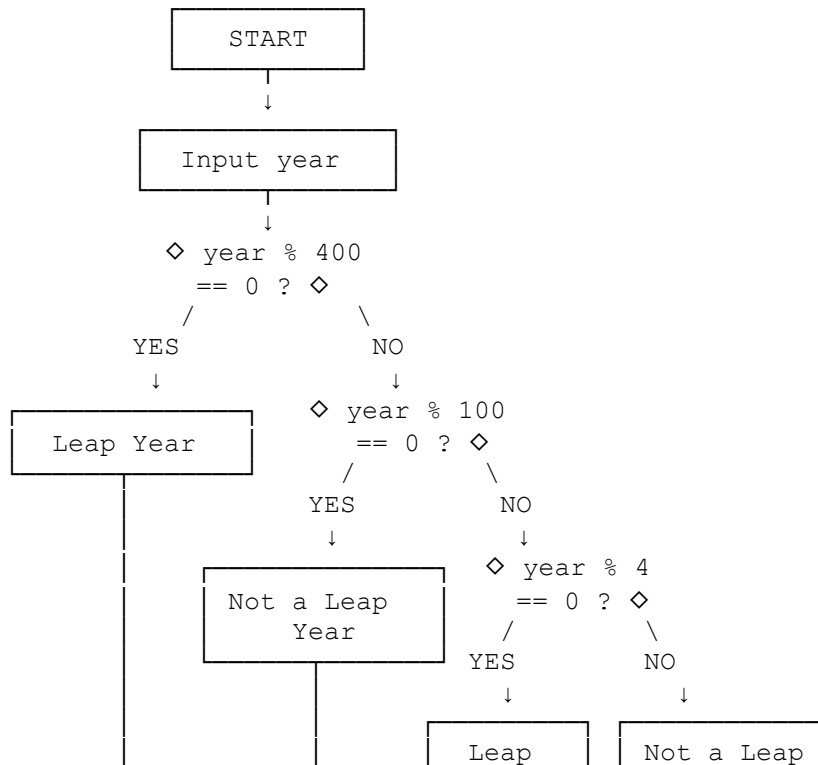
ELSE IF year % 4 = 0 THEN
    DISPLAY "Leap Year"

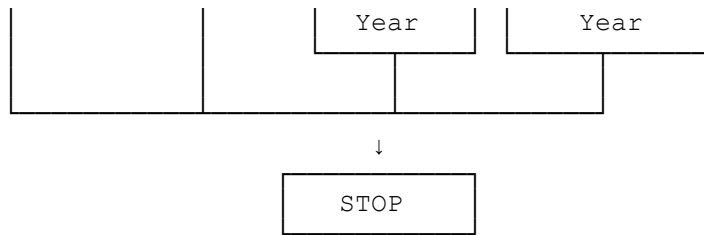
ELSE
    DISPLAY "Not a Leap Year"

END IF

END
```

5. Flowchart





6. C Program

```
#include <stdio.h>

int main()
{
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if (year % 400 == 0)
    {
        printf("%d is a Leap Year.\n", year);
    }
    else if (year % 100 == 0)
    {
        printf("%d is Not a Leap Year.\n", year);
    }
    else if (year % 4 == 0)
    {
        printf("%d is a Leap Year.\n", year);
    }
    else
    {
        printf("%d is Not a Leap Year.\n", year);
    }

    return 0;
}
```

7. Compile and Execute in Kali Linux

Save the program as leapyear.c.

```
cc leapyear.c
./a.out
```



```
(kali㉿kali)-[~/Desktop]
$ cc leapyear.c

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter a year: 2024
2024 is a Leap Year.

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter a year: 1900
1900 is Not a Leap Year.

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter a year: 2000
2000 is a Leap Year.
```

8. Evaluation Against Prepared Test Cases

Test Case 1 — 2024:

$2024 \% 4 = 0$ and it is not divisible by 100.

Actual output: **2024 is a Leap Year.**

Expected = Actual → **PASS ✓**

Test Case 2 — 1900:

$1900 \% 100 = 0$, but $1900 \% 400 \neq 0$.

Actual output: **1900 is Not a Leap Year.**

Expected = Actual → **PASS ✓**

Test Case 3 — 2000:

$2000 \% 400 = 0$.

Actual output: **2000 is a Leap Year.**

Expected = Actual → **PASS ✓**

Final Result

Both C programs compile successfully using the **cc compiler**, execute correctly, and **pass all three prepared test cases**.